Projet ANAC++ :

KOUADIO Akou Blaise Emmanuel Kevin

&

SONHAFO Mèsellius

Université de Lille - Département Mathématiques

Master 1 MATHEMATIQUE et APPLICATION :
Mathématiques Finance Computationnel, Actuariat

Encadrant : Madame Caterina CALGARO

2025-2026

26 octobre 2025

Table des matières

0.1 Avant-Propos 1
0.2 TP3 : Manipuler les matrices creuses (Sert pour le Projet TP4) 2
0.2.1 Tp3 exercice 1 : Stockage Profil 2
0.2.2 Tp3 exercice 2 : Stockage en bande 3
0.2.3 Tp3 exercice 3 : Stockage CSR 4
0.3 Projet TP 4 : Poisson, chaleur et Black-Scholes 5
0.3.1 Tp4 exercice 1 : Poisson 1D 5
0.3.2 Tp4 exercice 2 : Équation de la chaleur 7
0.3.3 Tp4 exercice 3 : Black-Scholes 9
0.4 Conclusion générale 11

0.1 Avant-Propos

Afin d'avoir une meilleure organisation et un travail plus structuré, ce travail a été centralisé en un fichier unique .cpp appelé : "projet_main_menu" qui contient tous les autres exercices du TP3 et du TP4. Ainsi l'exploration de chaque exercice sera plus facile et agréable.

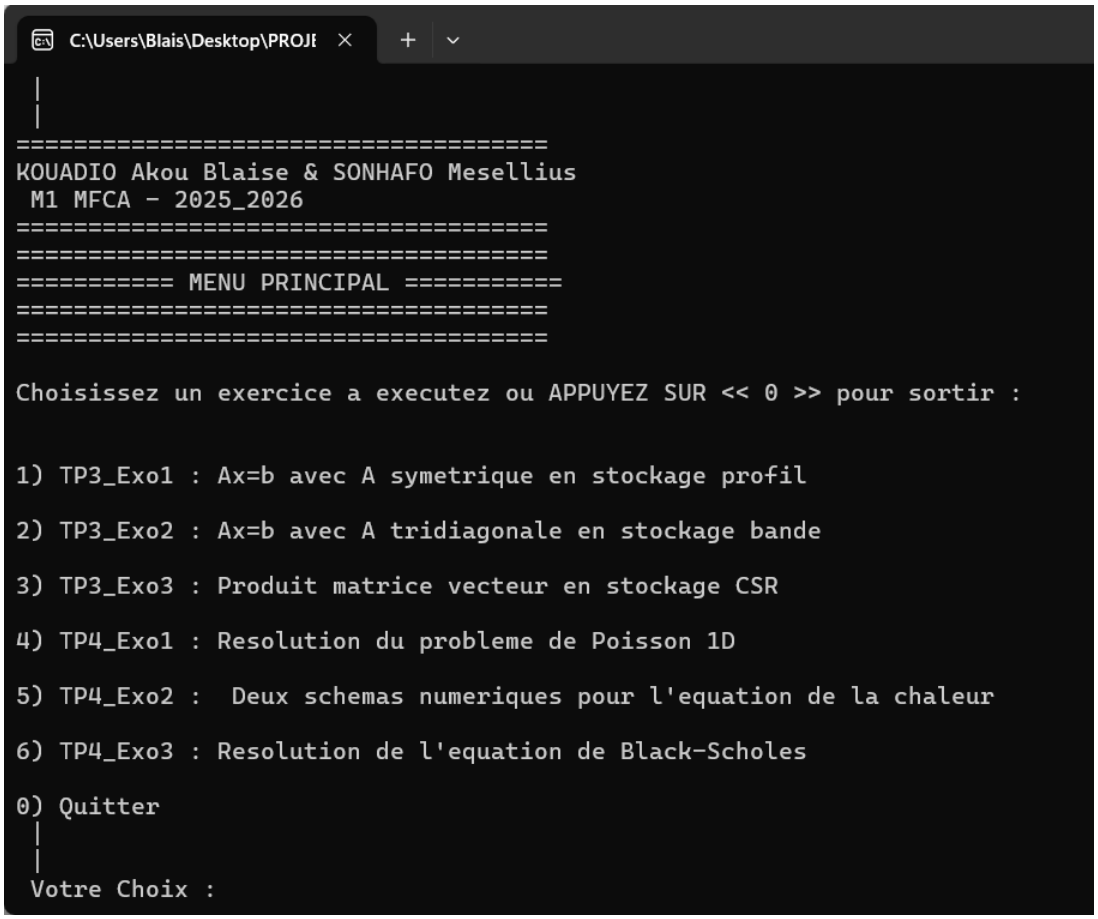


FIGURE 1 – Menu principale d'exploration des différents exercices.

Concernant la consultation des différents codes, ceux-ci sont répartis en fichiers .h et .cpp à travers différents dossiers, chacun inclus dans les dossiers TP3 et TP4 à la racine.
Étant donné que nous sommes sur Windows et ayant rencontré un problème pour la compilation sous Visual Studio Code, nous avons dû réaliser nos codes sous le logiciel : "Code::Block". Ainsi, en cas de non fonctionnement sur un système d'exploitation macOS ou Linux via Visual Studio Code, nous sommes invités à exécuter le programme sous "Code::Block".

0.2 TP3 : Manipuler les matrices creuses (Sert pour le Projet TP4)

0.2.1 Tp3 exercice 1 : Stockage Profil



FIGURE 2 – Résultats de l'exercice 1 du TP3.

0.2.2 Tp3 exercice 2 : Stockage en bande

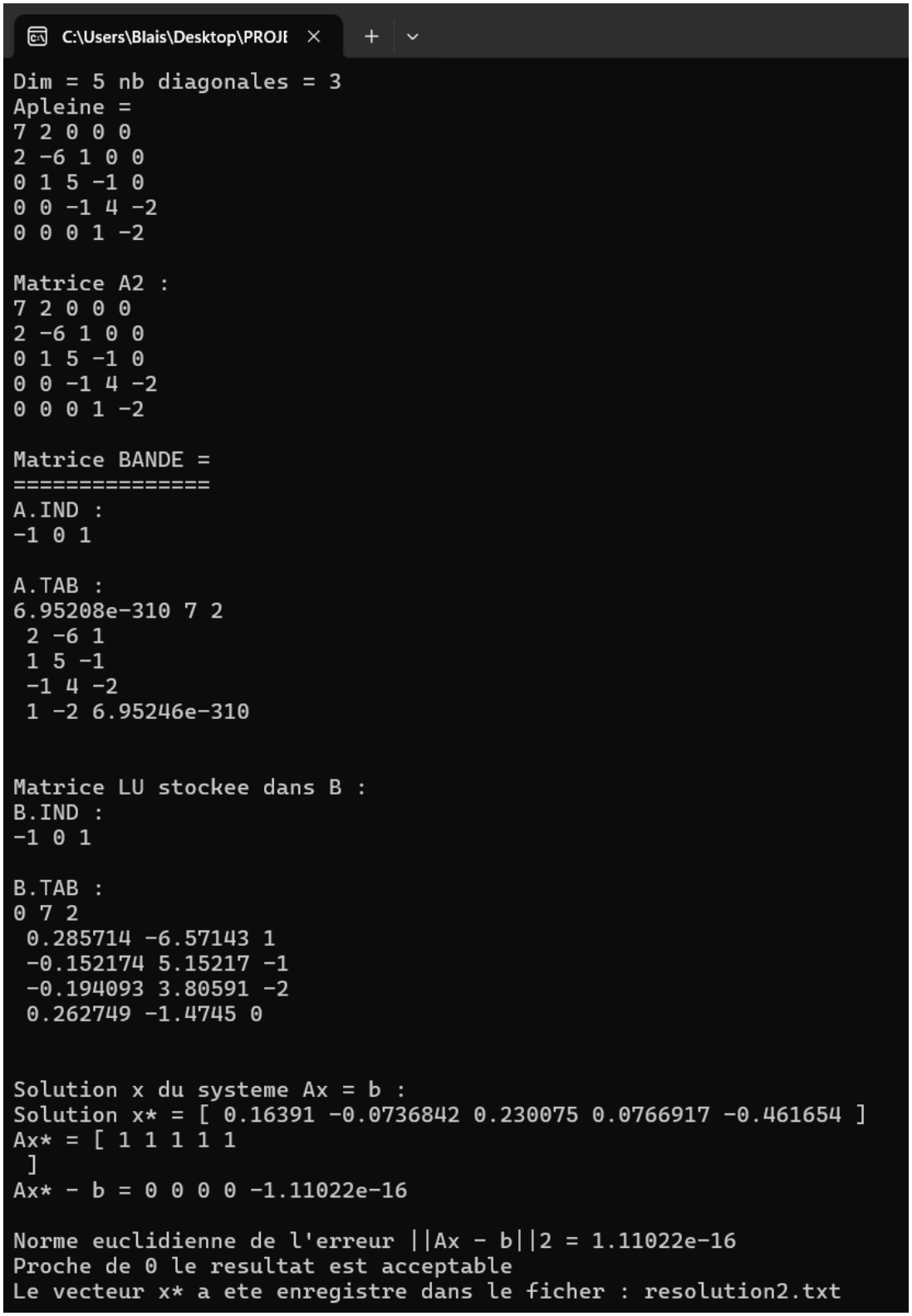


FIGURE 3 – Résultats de l'exercice 2 du TP3.

0.2.3 Tp3 exercice 3 : Stockage CSR

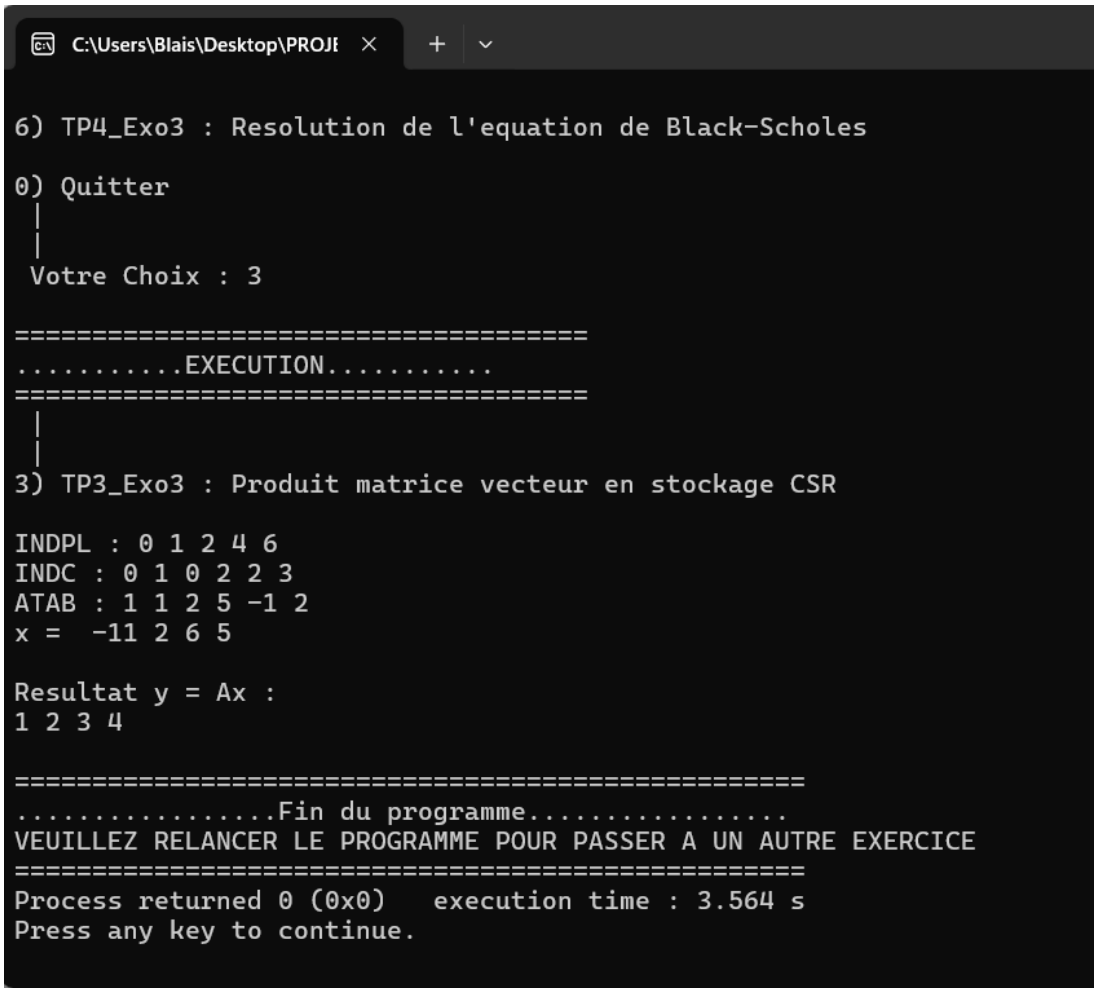


FIGURE 4 – Résultats de l'exercice 3 du TP3.

0.3 Projet TP 4 : Poisson, chaleur et Black-Scholes

0.3.1 Tp4 exercice 1 : Poisson 1D

```
C:\Users\Blais\Desktop\PROJE
 Votre Choix : 4

====================================
...........EXECUTION...........
====================================
 |
 |
4) TP4_Exo1 : Resolution du probleme de Poisson 1D

Poisson 1D periodique, alpha=0
     N         Erreur max
     4                inf
     8                inf
    16        4.005902e-01
    32        3.656039e-01
    64        1.230833e+00
   128        8.573733e-01
   256        1.741133e+00
   512        1.725475e-02
  1024        2.458594e-01
  2048        3.221928e-02
  4096        3.867244e-01
  8192        1.183664e+00
 16384        5.186072e-01
Poisson 1D periodique, alpha=1.000000e+00
     N         Erreur max
     4        2.266187e-01
     8        5.165156e-02
    16        1.262676e-02
    32        3.139192e-03
    64        7.837101e-04
   128        1.958596e-04
   256        4.896067e-05
   512        1.223990e-05
  1024        3.059966e-06
  2048        7.649950e-07
  4096        1.913014e-07
  8192        4.788688e-08
 16384        1.198609e-08

================================================
CONSULTEZ LES REPRESENTATIONS GRAPHIQUES DANS LE CODE PYTHON :
python_GRAPHES_TP4.ipynb
A LA RACINE DU FICHIER PROINCIPALE
================================================
.................Fin du programme.................
VEUILLEZ RELANCER LE PROGRAMME POUR PASSER A UN AUTRE EXERCICE
================================================
Process returned 0 (0x0)   execution time : 7.089 s
```

FIGURE 5 – Résultats de l'exercice 1 du TP4.

Ces résultats nous amène au graphique suivant depuis le code python "python_GRAPHE_TP4.ipynb" :

```
Taux de convergence (β) pour α=0: -0.079
Taux de convergence (β) pour α=1: -2.007
Coefficient R² pour α=0: 0.015
Coefficient R² pour α=1: 1.000
```

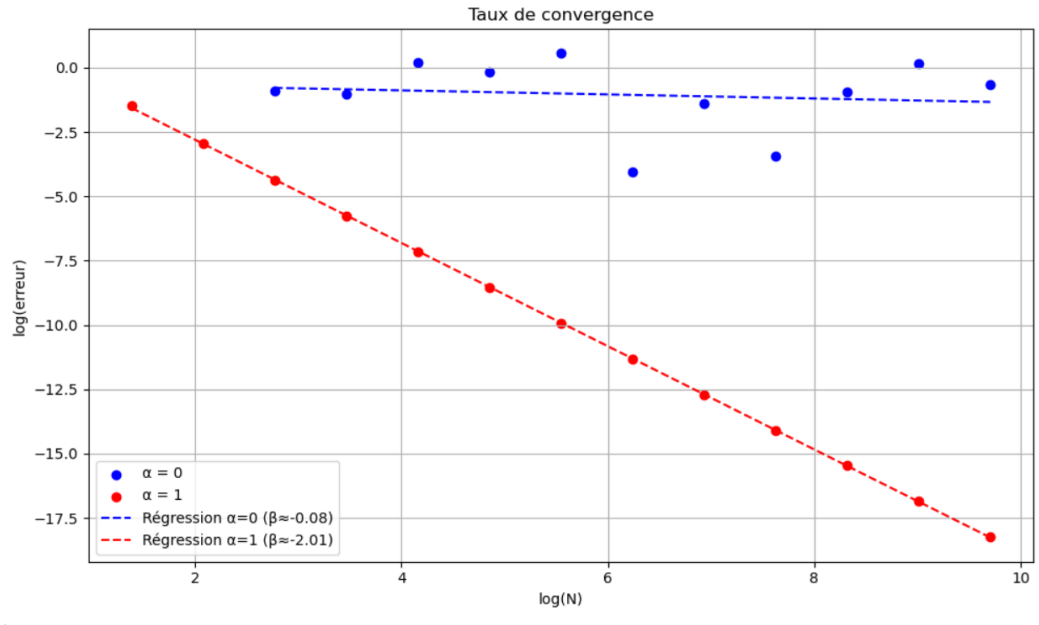


FIGURE 6 – Tp4_exo1 : Nuage $(\log(N))$, $(\log(\epsilon_N))$ et droite de régression pour $\alpha = 0$ et $\alpha = 1$.

Pour le calcul de f on a :
Soit $u_{\text{ex}}(x) = \sin(2\pi x)$ et $f(x) = -u''(x) + \alpha u(x)$.
On a $u'_{\text{ex}}(x) = 2\pi\cos(2\pi x)$ et $u''_{\text{ex}}(x) = -(2\pi)^2\sin(2\pi x) = -4\pi^2\sin(2\pi x)$.
Ainsi
$f(x) = -u''_{\text{ex}}(x) + \alpha u_{\text{ex}}(x) = (4\pi^2 + \alpha)\sin(2\pi x)$.

Concernant les résultats nous pouvons dire que :

Cas $\alpha = 1$: la régression en log par log a une pente $\beta = 2.01$ et un coefficient de détermination $R^2 = 1$. On a alors une convergence d'ordre 2, ce qui révèle un schéma aux différences centrées : $\|u - un\|_\infty \sim C\,h^2$.

Cas $\alpha = 0$: On a une pente $\beta = -0.08$ et un coefficient de détermination $R^2 = 0.15$ qui est très faible. On ne peut vraiment conclure sur une convergence car les points sont dispersés. En effet, quand on prend $\alpha = 0$, la matrice discrète dans les conditions périodiques est singulière. Ainsi, le système n'a pas de solution unique, car la solution est déterminée à une constante près. Et le système peut être incompatible numériquement selon la discrétisation.
Raison pour laquelle nous pouvons observer des erreurs infinies pour $N = 4$ ou $N = 4$ ou une suite d'erreurs non strictement monotone : dispersés.

En résumé pour $\alpha > 0$ on a une matrice semi définie positif, le le problème est bien posé et on observe une convergence d'ordre 2.
Pour $\alpha = 0$ la solution n'est pas unique et la méthode ne converge pas. Une correction possible dans ce cas serai de fixer $u(x_0) = 0$ car l'opérateur périodique $-u$" a un noyau non trivial.

0.3.2 Tp4 exercice 2 : Équation de la chaleur

```
C:\Users\Blais\Desktop\PROJE
 |
5) TP4_Exo2 :  Deux schemas numeriques pour l'equation de la chaleur

Veuillez choisir le schema que vous voulez executer :

 1) Schema explicite

 2) Schema implicite
|
|
Votre choix : 1
# Schema explicite, T=1.
 !!! Attention, l'execution du schema explicite peut etre longue quand N devient grand.

     N         Erreur max
     4  0.00636802839876
     8  0.00155237112703
    16  0.000385636584112
    32  9.73894440157e-05
    64  2.43370296883e-05
   128  6.08497726862e-06
   256  1.52141464299e-06

================================================
CONSULTEZ LES REPRESENTATIONS GRAPHIQUES DANS LE CODE PYTHON :
python_GRAPHES_TP4.ipynb
A LA RACINE DU FICHIER PROINCIPALE
================================================
.................Fin du programme.................
VEUILLEZ RELANCER LE PROGRAMME POUR PASSER A UN AUTRE EXERCICE
================================================
```

FIGURE 7 – Schéma explicite de l'exercice 2 du TP4.

```
C:\Users\Blais\Desktop\PROJE
 |
 |
5) TP4_Exo2 :  Deux schemas numeriques pour l'equation de la chaleur

Veuillez choisir le schema que vous voulez executer :

 1) Schema explicite

 2) Schema implicite
|
|
Votre choix : 2
# Schema implicite (LU sur mat_bande), T=1
     N         Erreur max
     4  0.00648797597336
     8  0.00182312361618
    16  0.00057957403703
    32  0.000246310855793
    64  0.000139508604729
   128  7.46590569736e-05
   256  3.86032206515e-05

================================================
CONSULTEZ LES REPRESENTATIONS GRAPHIQUES DANS LE CODE PYTHON :
python_GRAPHES_TP4.ipynb
A LA RACINE DU FICHIER PROINCIPALE
================================================
.................Fin du programme.................
VEUILLEZ RELANCER LE PROGRAMME POUR PASSER A UN AUTRE EXERCICE
================================================
```

FIGURE 8 – Schéma implicite de l'exercice 2 du TP4.

Ces résultats nous amène au graphique suivant depuis le code python "python_GRAPHE_TP4.ipynb" :

```
Taux de convergence (β) schéma explicite: -2.002
Taux de convergence (β) schéma implicite: -1.195
Coefficient R² explicite: 1.000
Coefficient R² implicite: 0.974
```

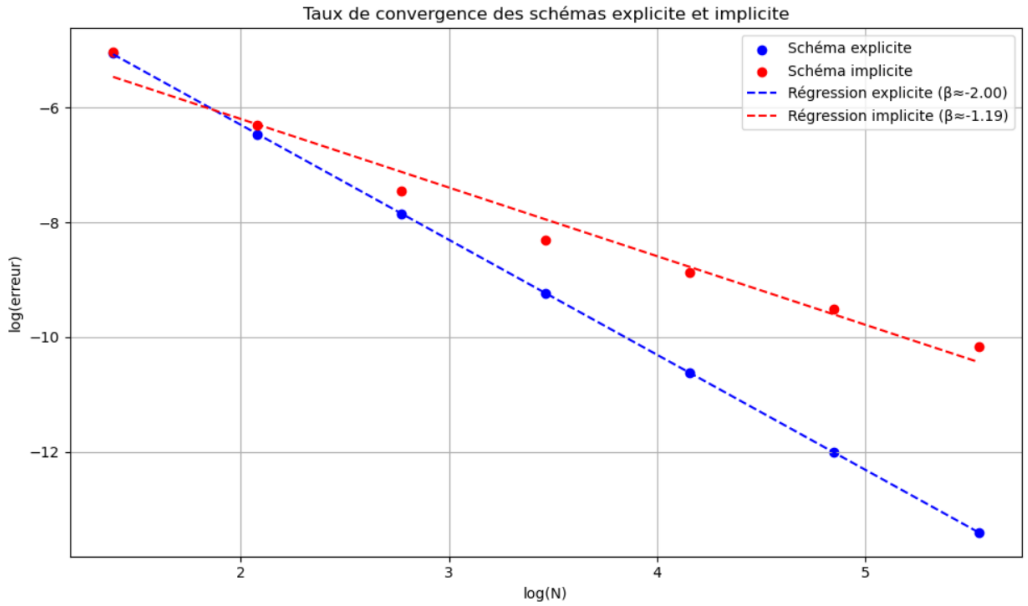


FIGURE 9 – Tp4_exo2 : Nuage $(\log(N))$, $(\log(\epsilon_N))$ et droite de régression pour $\alpha = 0$ et $\alpha = 1$.

Ici pour implémenter le schéma explicite l'on ne pouvait utiliser la factorisation de Crout LDL^T : Dans le cas où $\nu(x)$ est variable (voire discontinue), la discrétisation implicite de $u_t - \nu(x)\, u_{xx} = f$ donne, à la ligne i,

$$\left(1 + \frac{2k}{h^2}\nu(x_i)\right) U_i^{n+1} - \frac{k}{h^2}\nu(x_i)\, U_{i-1}^{n+1} - \frac{k}{h^2}\nu(x_i)\, U_{i+1}^{n+1} = \cdots$$

d'où les coefficients de la matrice A :

$$a_{i,i} = 1 + \frac{2k}{h^2}\nu(x_i), \qquad a_{i,i-1} = -\frac{k}{h^2}\nu(x_i), \qquad a_{i,i+1} = -\frac{k}{h^2}\nu(x_i).$$

Mais, sur la ligne $i-1$, on a

$$a_{i-1,i} = -\frac{k}{h^2}\nu(x_{i-1}),$$

si bien que, si $\nu(x_{i-1}) \neq \nu(x_i)$, alors

$$a_{i-1,i} \neq a_{i,i-1}.$$

Ainsi A est tridiagonale mais **non symétrique** (sauf si ν est constante), et l'on ne peut pas appliquer la factorisation de Crout $LDL^\top$, qui exige une matrice symétrique (définie positive dans notre cas). On utilisera donc plutôt une factorisation LU (avec pivot partiel).

Remarque : si l'on discrétise la forme conservative $(\nu u_x)_x$ avec des valeurs d'interface $\nu_{i\pm\frac{1}{2}}$, on retrouve une matrice symétrique, et l'on pourrait alors employer $LDL^\top$ (ou Cholesky).

Comme commentaire des résultats graphiques nous pouvons dire que :

Schéma explicite : La régression donne $\beta = -2.00$ avec $R^2 = 1.00$. On observe donc une convergence d'ordre 2 en fonction de N : $\log(\text{err}) \approx a + \beta \log N$ avec $\beta \approx -2$, soit $\text{err} \approx C\,N^{-2} \equiv C\,h^2$ puisque $h \sim 1/N$. Ceci est cohérent avec un pas de temps vérifiant la condition de stabilité, par exemple $k = \mathcal{O}(h^2)$, qui met l'erreur temporelle $\mathcal{O}(k)$ au même ordre que l'erreur spatiale $\mathcal{O}(h^2)$.

Schéma implicite : La régression donne $\beta \simeq -1.20$ avec $R^2 \simeq 0.97$, ce qui indique une convergence proche de l'ordre 1 : $\text{err} \approx C\,N^{-1} \equiv C\,h$. Le schéma d'Euler implicite est d'ordre 1 en temps ; si l'on prend k proportionnel à h, l'erreur temporelle $\mathcal{O}(k)$ domine l'erreur spatiale $\mathcal{O}(h^2)$, d'où la pente observée ≈ -1 sur le graphe $\log(\text{err})$–$\log N$.

En résumé pour le schéma explicite sous k proportionnel à h, on voit l'ordre 2 global : $\text{err} = \mathcal{O}(h^2)$. Pour le schéma implicite avec k proportionnel à h, l'ordre est gouverné par le temps (ordre 1). Pour obtenir l'ordre 2 en implicite, on peut soit prendre $k \propto h^2$, soit utiliser un schéma en temps d'ordre 2.

0.3.3 Tp4 exercice 3 : Black-Scholes

```
C:\Users\Blais\Desktop\PROJI
 |
6) TP4_Exo3 : Resolution de l'equation de Black-Scholes

Parametres: N=400  L=10000  T=365  sigma=0.2
h = 25.000000, k = 25.000000, M (pas de temps) = 15
Echantillons u(T,x):
x=0.000000  u(T,x)=0.000000
x=1000.000000  u(T,x)=4.742215
x=2000.000000  u(T,x)=6.351443
x=3000.000000  u(T,x)=7.298948
x=4000.000000  u(T,x)=7.918861
x=5000.000000  u(T,x)=8.340189
x=6000.000000  u(T,x)=8.627876
x=7000.000000  u(T,x)=8.819586
x=8000.000000  u(T,x)=8.939223
x=9000.000000  u(T,x)=9.002943
x=10000.000000  u(T,x)=9.022186

Fichiers ecrits:
 - u_final.csv
 - u_surface.csv

================================================
CONSULTEZ LES REPRESENTATIONS GRAPHIQUES DANS LE CODE PYTHON :
python_GRAPHES_TP4.ipynb
A LA RACINE DU FICHIER PROINCIPALE
================================================
.................Fin du programme.................
VEUILLEZ RELANCER LE PROGRAMME POUR PASSER A UN AUTRE EXERCICE
================================================
```

FIGURE 10 – Résultats de l'exercice 3 du TP4.

Ces résultats nous amène au graphique suivant depuis le code python "python_GRAPHE_TP4.ipynb" :

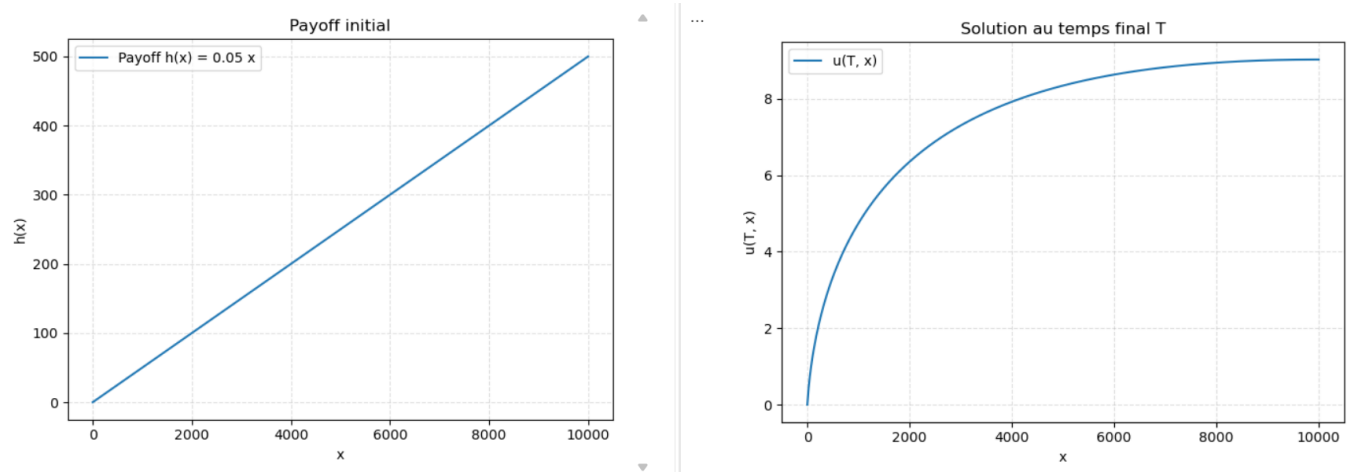


FIGURE 11 – Tp4_exo3 : Payoff initial et Solution au temps final T

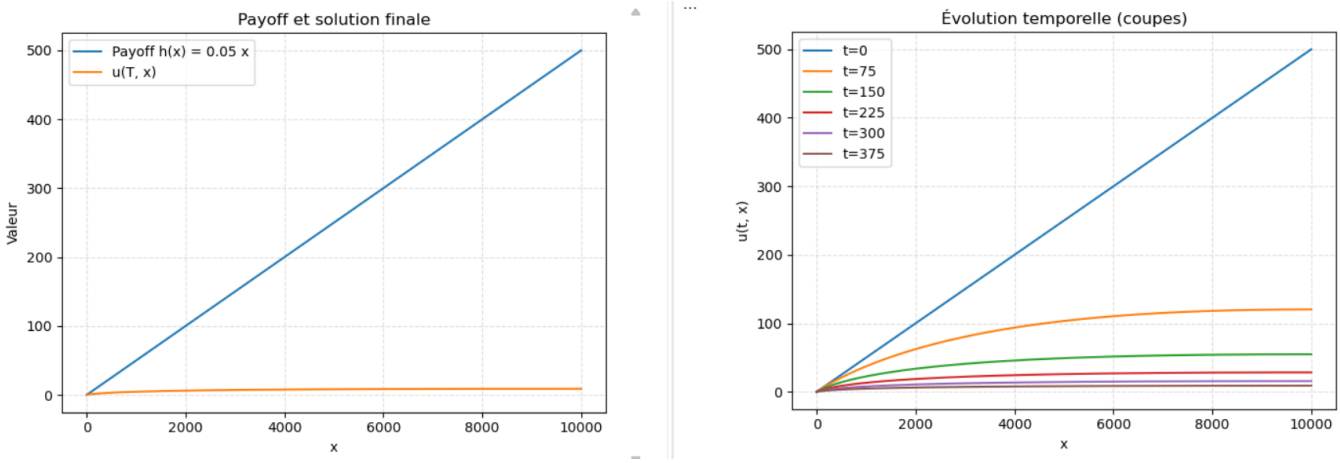


FIGURE 12 – Tp4_exo3 : Payoff initial et Solution au temps final en évolution temporelle

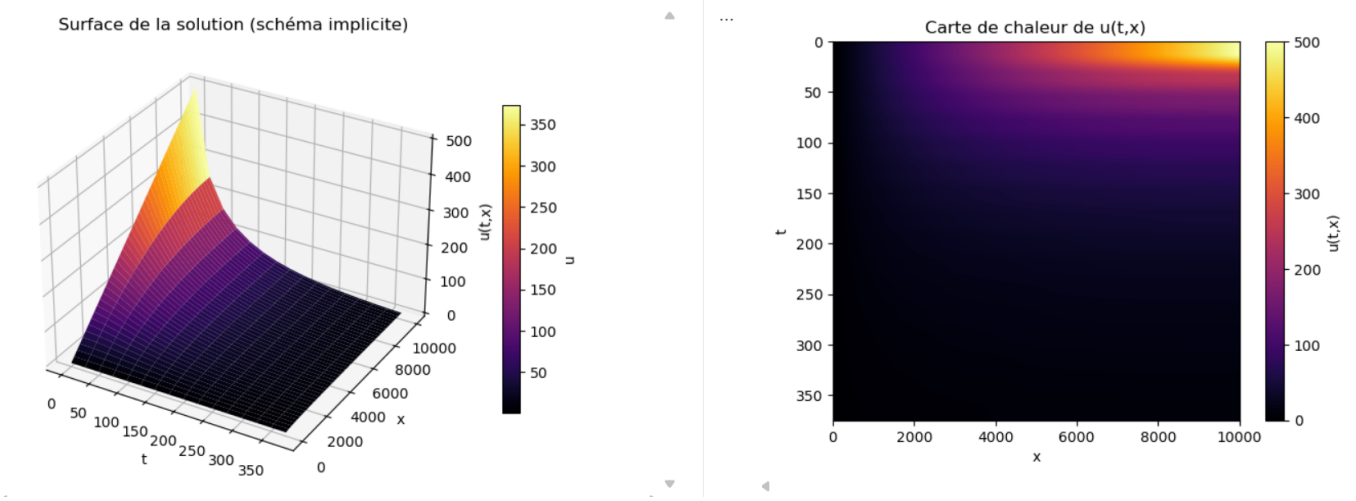


FIGURE 13 – Tp4_exo3 : Représentation 3D en surface et en carte thermique

Comme commentaire nous pouvons dire que :

(1) Surface 3D $u(t,x)$ **:** $u(t,x)$ est le *prix* de l'option en fonction du temps t et du cours x. La surface est **plus élevée** pour les grands x ainsi une option vaut plus quand l'actif monte et **baisse**

avec t : c'est l'érosion de la valeur temps, le temps qui passe fait perdre de la valeur.

(2) Carte de chaleur $u(t,x)$ **:** Les couleurs chaudes près de $t = 0$ montrent que le prix est plus élevé au départ, puis **diminue au fil du temps**. Près de $x = 0$, la valeur reste proche de 0, une option d'achat vaut presque rien si l'actif vaut presque 0. Près de $x = L$, la pente devient faible (condition de Neumann $u_x(t, L) = 0$) : *très haut* sur l'actif, le prix **se stabilise**, la sensibilité Δ tend vers 0 sur la borne.

(3) Payoff initial $h(x)$ **:** C'est ce qu'on recevrait à l'échéance *sans aléa* : ici $h(x) = 0.05\,x$, donc **linéaire et croissant**. Au temps initial du calcul ($t = 0$ dans notre temps inversé), on part de cette courbe.

(4) Solution finale $u(T,x)$ **:** Après avoir laissé évoluer le modèle avec la volatilité σ, la courbe finale $u(T,x)$ est **en dessous** du payoff initial : la *valeur temps* a disparu et la *diffusion* (l'aléa) a **lissé** la courbe. Pour de très grands x, le prix **sature** (pente ≈ 0) à cause de $u_x(T, L) = 0$.

(5) Payoff vs prix final : En superposant $h(x)$ et $u(T,x)$, on voit clairement que $0 \leq u(T,x) \leq h(x)$ pour tout x : le **prix d'aujourd'hui** est **inférieur** au gain certain à l'échéance, il reste du risque et du temps à parcourir. Plus x est grand, plus l'effet de la volatilité σ ou diffusion **aplatit** la courbe.

(6) Coupes temporelles : Les coupes $t = 0, 75, 150, \ldots$ montrent la **décroissance régulière** du prix dans le temps et un **lissage progressif** : au début, les courbes sont proches du payoff $h(x)$, puis elles se **rapprochent d'un profil plus plat** en avançant dans le temps. En bas à gauche, $u(t, 0) \approx 0$ (l'option vaut presque rien si x est très bas) ; en haut à droite, $u_x(t, L) \approx 0$, sensibilité quasi nulle sur la borne.

Pour synthétiser : plus x est grand, plus l'option **vaut** c'est un effet directionnel ; plus t avance, plus la valeur **diminue** : érosion du temps. La volatilité σ **lisse** les prix au cours du temps.

0.4 Conclusion générale

En synthèse, lorsque le problème est bien posé et que la stabilité est respectée, les schémas centraux délivrent l'ordre attendu ($\mathcal{O}(h^2)$). Le choix de k *relativement* à h détermine l'ordre observé, et les graphes confirment ces diagnostics analytiques.
Ce TP nous a permis de mettre en œuvre des techniques numériques variées pour la solution d'équations aux dérivées partielles. Les schémas étudiés ont démontré une convergence conforme à la théorie et une stabilité adaptée aux contraintes numériques. Le modèle de Black-Scholes a illustré la transposition de ces méthodes à la finance mathématique.
Pour notre part, le problème commun que nous avons rencontré lors de la réalisation de ce travail était l'utilisation des fonctions : factorisation_LDLt(A) et resolution_LDLt(A,b), factorisation_lu et resolution_lu développées au TP3 afin de les appliquer aux exercices du TP4. Nous avons dû les modifier par rapport aux algorithmes du cours afin d'avoir les résultats.